

The Constant Evolution of C#



Mike Woodring

Programmer | Learner | Teacher

@mcwoodring | [linkedin.com/in/woodring](https://www.linkedin.com/in/woodring)



Heraclitus (of Ephesus) – common translation

“The only constant is change.”



Heraclitus (of Ephesus) – literal translation

“Life is flux.”



Principles of C# Evolution (Observed)

Incremental, not abrupt

Opt-in, not mandatory

Nonbreaking, not disruptive

Codification of common patterns & best practices that emerge in the community

Simpler and/or faster and/or safer code



The Constant Evolution of C#



Mike Woodring

Programmer | Learner | Teacher

@mcwoodring | [linkedin.com/in/woodring](https://www.linkedin.com/in/woodring)



The Conventional Evolution of C#



Mike Woodring

Programmer | Learner | Teacher

@mcwoodring | [linkedin.com/in/woodring](https://www.linkedin.com/in/woodring)





C# Evolution – a Sampling



Main – Classic

```
class Program
{
    static void Main()
    {
        System.Console.WriteLine("Hello, world!");
    }
}
```

```
c:\> program.exe
```

```
Hello, world!
```

Main – Top Level Statements

```
System.Console.WriteLine("Hello, world!");
```

```
c:\> program.exe
```

```
Hello, world!
```

Main – with Command Line Args

```
using System;

Console.WriteLine("Hello, world!");

for (var n = 0; n < args.Length; n++)
{
    Console.WriteLine($"args[{n}] = {args[n]}");
}

return 0;
```

```
c:\> program.exe C# rocks
```

```
Hello, world!
args[0] = C#
args[1] = rocks
```

Canonical Properties - Before

Point.cs

```
public class Point
{
    private int _x;
    private int _y;

    public int X
    {
        get { return _x; }
        set { _x = value; }
    }

    public int Y
    {
        get { return _y; }
        set { _y = value; }
    }

    public override string ToString() {
        return $"({X}, {Y})";
    }
}
```

Program.cs

```
var pt = new Point { X = 30, Y = 12 };
System.Console.WriteLine(pt.ToString());
```

Canonical Properties - After

Point.cs

```
public class Point
{
    public int X { get; set; }
    public int Y { get; set; }

    public override string ToString() {
        return $"({X}, {Y})";
    }
}
```

Program.cs

```
var pt = new Point { X = 30, Y = 12 };
System.Console.WriteLine(pt.ToString());
```

Init Properties

Point.cs

```
public class Point
{
    public int X { get; init; }
    public int Y { get; init; }

    public override string ToString() {
        return $"({X}, {Y})";
    }
}
```

Program.cs

```
var pt = new Point { X = 30, Y = 12 };
System.Console.WriteLine(pt.ToString());
```

Required Init Properties

Point.cs

```
public class Point
{
    public int X { get; init; }
    public int Y { get; init; }

    public override string ToString() {
        return $"({X}, {Y})";
    }
}
```

Program.cs

```
var pt = new Point { Y = 12 };
System.Console.WriteLine(pt.ToString());
```

Required Init Properties

Point.cs

```
public class Point
{
    public required int X { get; init; }
    public required int Y { get; init; }

    public override string ToString() {
        return $"({X}, {Y})";
    }
}
```

Program.cs

```
var pt = new Point { Y = 12 };
System.Console.WriteLine(pt.ToString());
```

Expression-bodied Members

```
public class Point
{
    public required int X { get; init; }
    public required int Y { get; init; }

    public override string ToString() {
        return $"{X}, {Y}";
    }
}
```



Expression-bodied Members

```
public class Point
{
    public required int X { get; init; }
    public required int Y { get; init; }

    public override string ToString() => $"({X}, {Y})";
}
```



Collection Initializers

```
string[] stooges = new string[] {"Moe", "Larry", "Curly"};

foreach (var s in stooges)
{
    Console.WriteLine(s);
}
```

```
c:\> program.exe
```

```
Moe
Larry
Curly
```

Collection Initializers

```
string[] stooges = new [] {"Moe", "Larry", "Curly"};

foreach (var s in stooges)
{
    Console.WriteLine(s);
}
```

```
c:\> program.exe
```

```
Moe
Larry
Curly
```

Collection Expressions

```
string[] stooges = ["Moe", "Larry", "Curly"];  
  
foreach (var s in stooges)  
{  
    Console.WriteLine(s);  
}
```

```
c:\> program.exe
```

```
Moe  
Larry  
Curly
```

Collection Spread Operator

```
string[] twoStooges = ["Moe", "Larry"];  
string[] moreStooges = ["Curly", "Shemp"];  
string[] fourStooges = [..twoStooges, ..moreStooges];
```

```
foreach (var s in fourStooges)  
{  
    Console.WriteLine(s);  
}
```

```
c:\> program.exe
```

```
Moe  
Larry  
Curly  
Shemp
```

Relational Patterns – Before

Point.cs

```
enum Generation { BabyBoomer, GenX, Millenial, GenZ, GenAlpha, GenBeta }

class Person
{
    public required int BirthYear { get; init; }

    public Generation Generation
    {
        get
        {
            if ((BirthYear >= 1946) && (BirthYear <= 1964))
            {
                return Generation.BabyBoomer;
            }
            else if ((BirthYear >= 1965) && (BirthYear <= 1980))
            {
                return Generation.GenX;
            }
            else if ((BirthYear >= 1981) && (BirthYear <= 1996))
            {
                return Generation.Millenial;
            }
            else if ((BirthYear >= 1997) && (BirthYear <= 2012))
            {
                return Generation.GenZ;
            }
            else if ((BirthYear >= 2013) && (BirthYear <= 2024))
            {
                return Generation.GenAlpha;
            }
            else
            {
                return Generation.GenBeta;
            }
        }
    }
}
```

Program.cs

```
var p = new Person { BirthYear = 1970 };
System.Console.WriteLine(p.Generation);
```

Relational Patterns - After

```
enum Generation { BabyBoomer, GenX, Millenial, GenZ, GenAlpha, GenBeta }

class Person
{
    public required int BirthYear { get; init; }

    public Generation Generation => BirthYear switch {
        (>= 1946) and (<= 1964) => Generation.BabyBoomer,
        (>= 1965) and (<= 1980) => Generation.GenX,
        (>= 1981) and (<= 1996) => Generation.Millenial,
        (>= 1997) and (<= 2012) => Generation.GenZ,
        (>= 2013) and (<= 2024) => Generation.GenAlpha,
        _ => Generation.GenBeta
    };
}
```



Summary



Principles

- Incremental
- Non-breaking
- Opt-in
- Simpler and/or safer and/or faster



Courses Referenced



[C# Fundamentals](#) by Gill Cleeren



[C# Advanced Language Features](#) by Filip Ekberg



Documentation



<https://docs.microsoft.com/dotnet/>



<https://docs.microsoft.com/dotnet/csharp/>



<https://devblogs.microsoft.com/dotnet/category/csharp/>



Platforms & Community



<https://github.com/dotnet/core>



<https://github.com/dotnet/csharpplang>



<https://github.com/dotnet/csharpplang/discussions>



<https://dotnet.microsoft.com/platform/community>



C#: The Big Picture



Approachable (to C++ & Java devs)

Managed (with help from the CLR)

Resilient & safe, with native performance

Acquired BCL skills are transferable

Conventional evolution is to be expected

